Making a pinball game for Playdate: Part 02, the physics

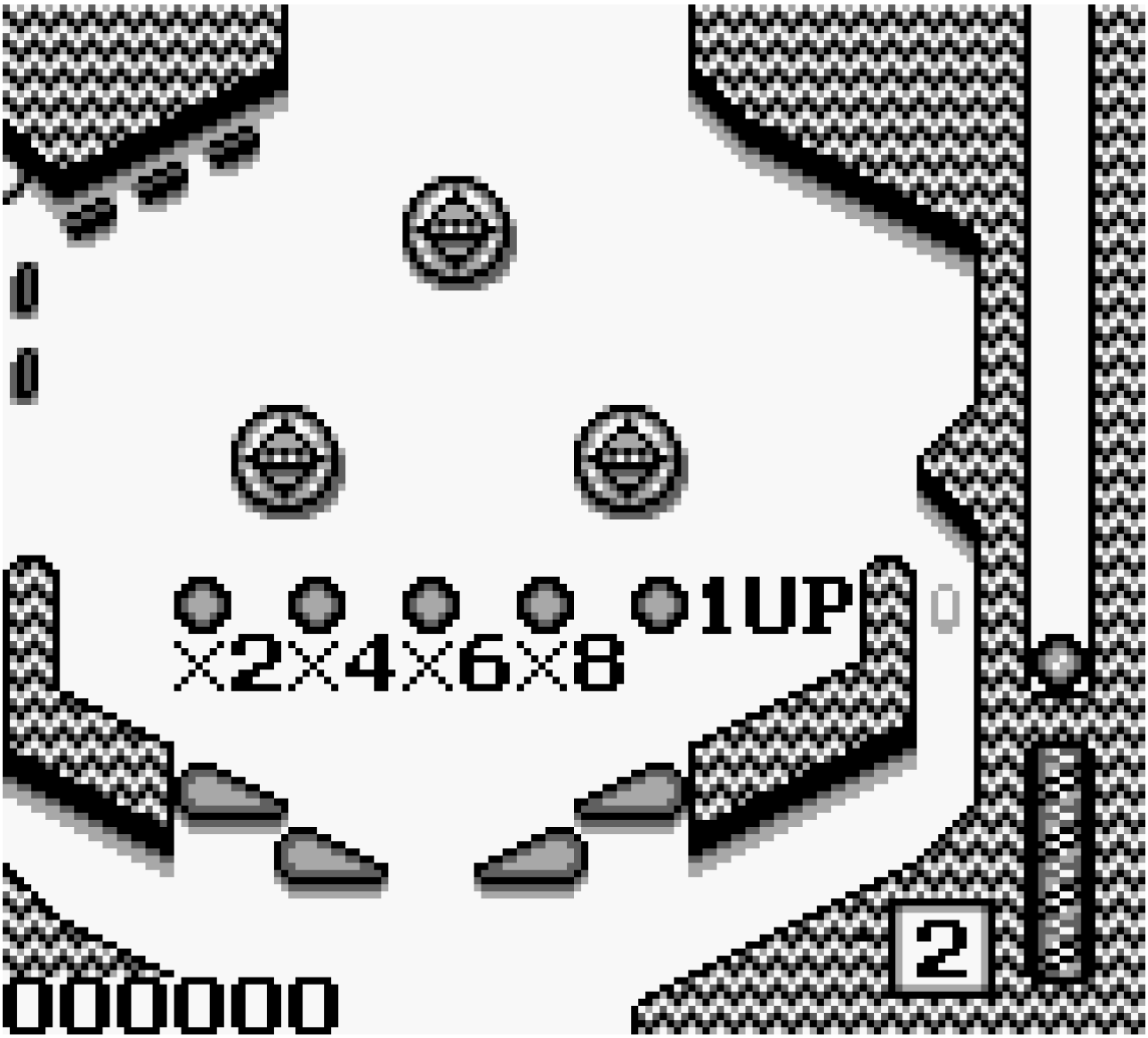
Monday, December 2, 2024

Source: https://amano.games/devlog/making-a-pinball-game-for-the-playdate-part-02-the-physics

Welcome to this December adventure, where I will try to write about the process of our latest game, Devils on the Moon pinball.

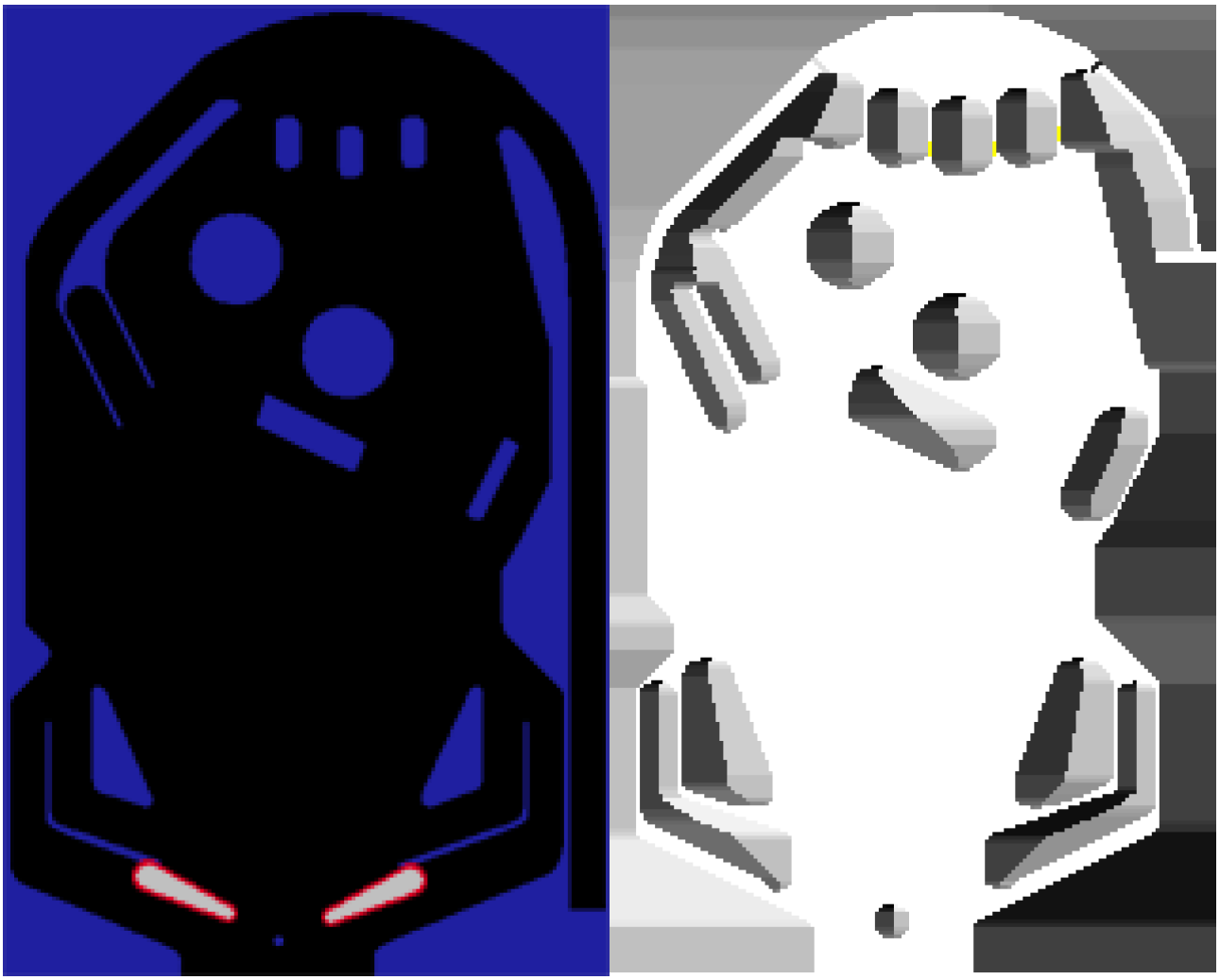
When we decided to make a *pinball* game, the main challenge I saw was the physics system. I don't consider myself to be someone good at math, and physics simulations sound like a scary big math mountain to climb.

But there was always a small sliver of hope; old *Game Boy pinball* games didn't look like they had a really sophisticated physics simulation, so If we at least managed something like that, we would be fine.



I started looking more into how these games worked. I found a couple of good answers online.

The main gist of it seemed to be to generate a collision mask where every pixel had encoded the value of the angle the ball had to bounce off in a value between 0-255. You then read the value of each pixel around the radius of the ball, get the x and y components of the force vector from the average of the angle, and apply the force to the ball. This sounds... simple!

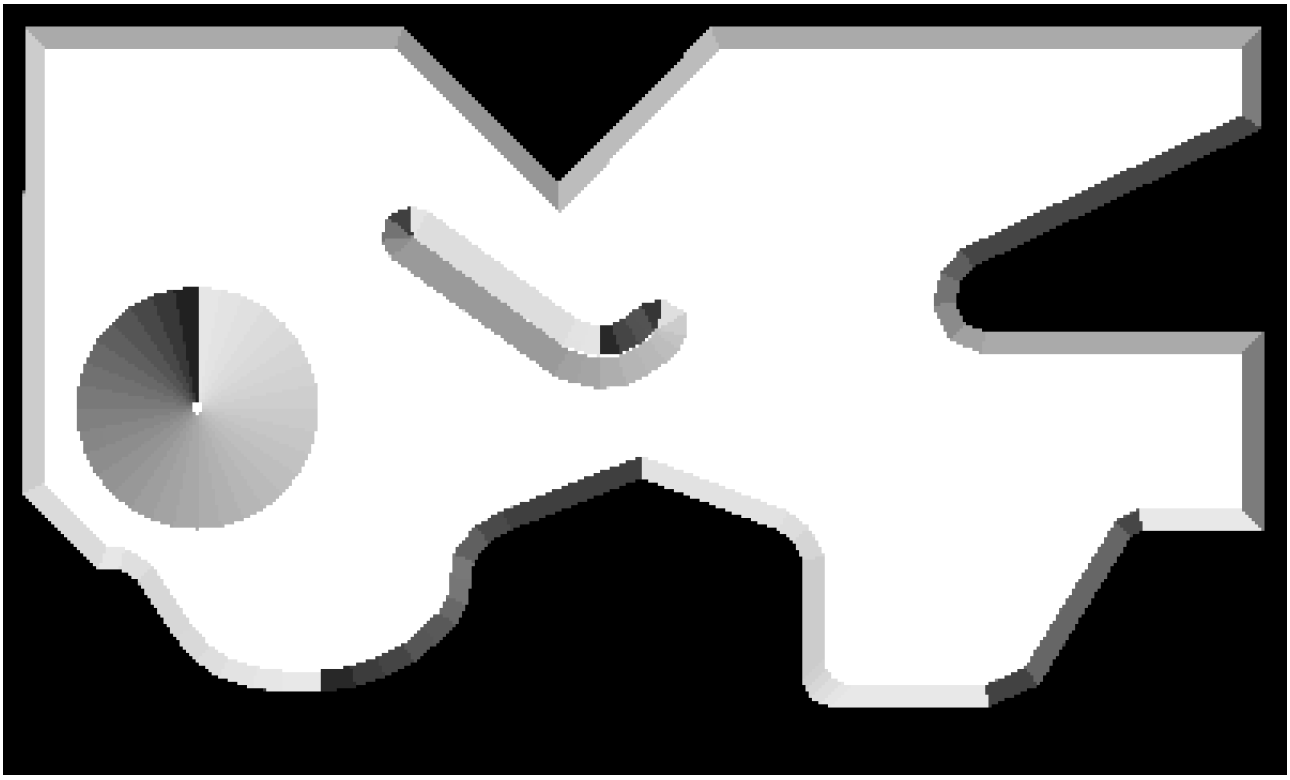


One thing that you need to understand is that here at **Amano**, after 10 years working together, **Jp** and I try to always work in parallel. Gone are the days were **JP** would send me the file through Dropbox, I would put them in the correct folder with the correct naming convention, show him my build of the game, tweak a couple of values and then recompile.

My main priority while making a system for a game we will do together is to give **JP** the tools to be able to work on design while I'm working on the next system.

So going back to the collision mask, I just didn't know a good way of generating these masks. Today, a year later, I have some ideas, but at the time I felt there was not enough information around on how this was supposed to work.

JP who has become steadily better at 3D over the years, saw it as an opportunity to use Blender and came up with a pipeline on how to generate a mask like that.



I tried it for a day but quickly became frustrated; for one, as I said, there was little information on how to do this. I didn't understand the math behind it, as simple as it was, and I depended on **JP** to generate a new image if I wanted to try a new mask.

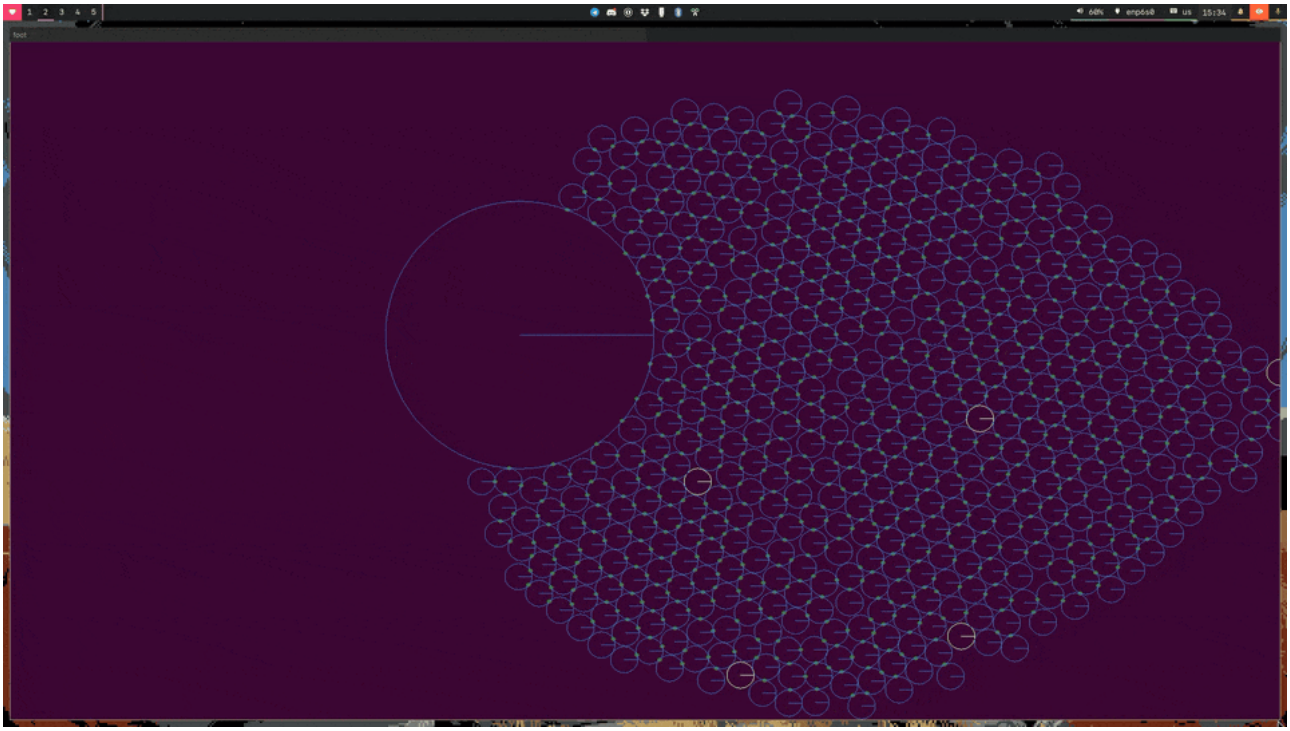
I researched a little on the state of physics libraries for Playdate, but the results weren't encouraging. There were a couple of bindings for Box2D and Chipmunk physics, but the libraries seemed too complicated for what I needed they didn't seemed to have the results I wanted.

I was kind of lost; on one hand, I could try to figure out the collision mask method and have the simple janky physics of some early *Game Boy pinball* games. And in the other, I could just try to do a rigid body simulation myself and see how that goes.

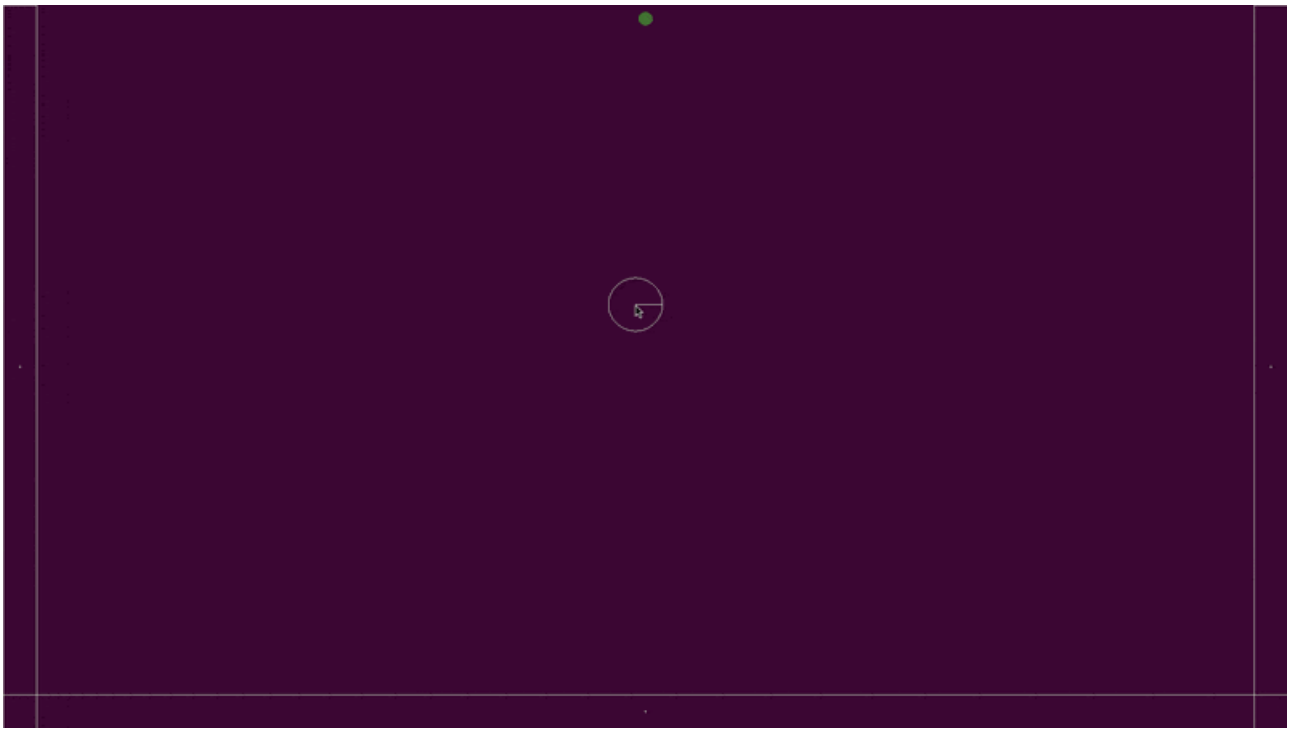
My first rule when I started this project was, "I prefer something simpler and done by myself, rather than doing something more complicated and just grabing someone else's code." So going for the collision mask seemed like the best option if I was going for simplicity, but the lack of resources made it harder on the ‘me understanding it’ part of the rule.

For a long time I had been wanting to do one of the courses on Pikuma.com. They seemed to share a similar spirit to Hand Made Hero, and they had a course on a 2D physics engine, just what I needed, neat!

I skimmed the contents, and it seemed like just the thing I needed.



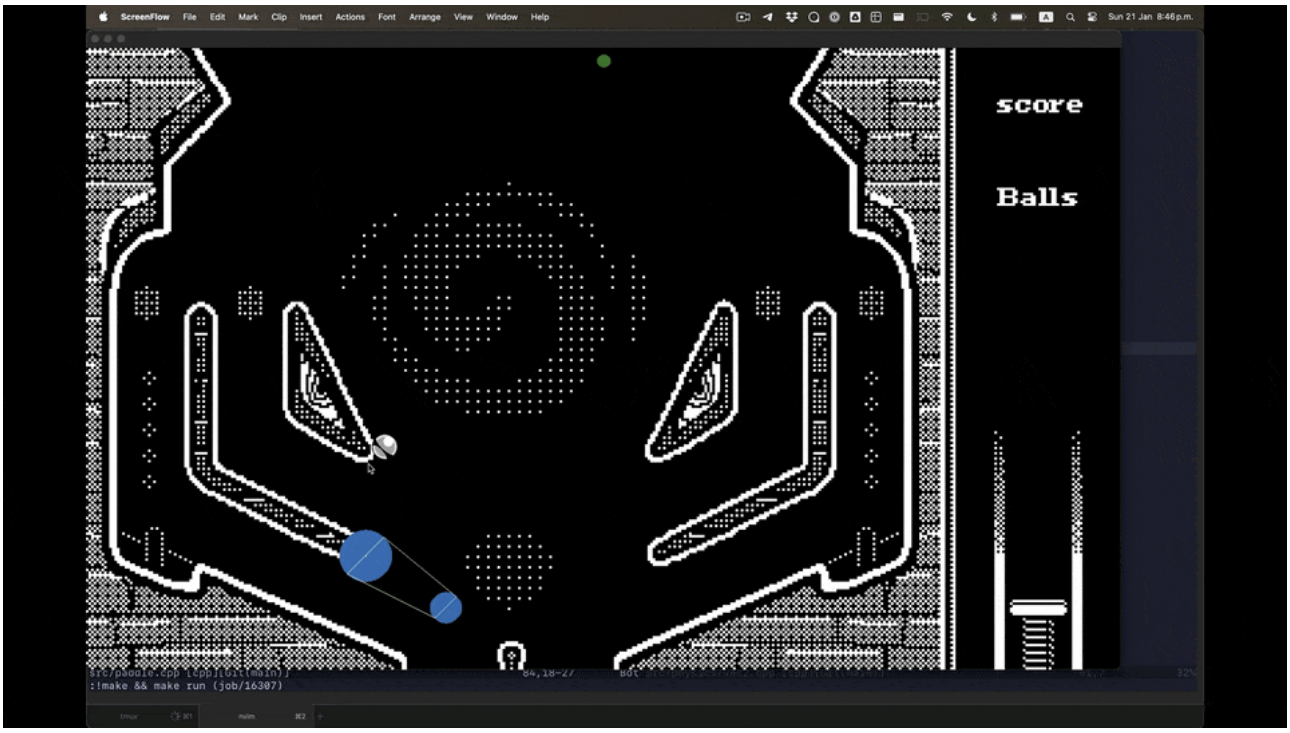
It took me around a week to finish the course and it was good! I don't understand all the underlying math and wouldn't be able to derivate the formulas by myself, but I understood enough to feel confident to reimplement the physics engine in *C* for our *pinball* game.



It was a *dangerous* decision because while I took a full week learning new things, I left **JP** with the hype of a new project and not a lot of tools to work with, but after showing my progress, we were both excited.

The main benefit of doing things ourselves meant that we could cut corners where it made sense for our specific project.

Now I had to port my new physics engine to C and make it work on the Playdate so we could validate how many things we could have on the screen at the same time.



But first I started to worry. Remember how I said that my main priority was having tools ready so that **JP** could work in parallel while I worked on the next thing? Authoring the collisions for the game started to look like a task more difficult than what I was expecting.

See you in the next adventure, Part 03: The first level editor.

Resources

Before I go go. I want to share a couple of links to resources that helped me tremendously with this. And on that note, I also want to tell you that doing a basic rigid body simulator is not that hard! If you are even a bit interested, you should try it! I don't regret it one bit, and you can have something fun happen quite fast.

How to create a custom physics engine by Randy Gaul

By far the easiest of the bunch, you can follow along and have something working in a day, also Randy Gaul so if you haven't checked their libraries of CuteHeaders I highly recommend them.

Game physics series by Allen Chou

A little bit more in depth and touch on more intermediate subjects, whenever I got stuck with a specific concept, I would try to read this article series, a lot of times it went over my head! But the little knowledge that I was able to absorb was super valuable.

Rigid Body Dynamics by Chris Hecker

Chris Hecker was mentioned quite a lot in the Hand Made Hero video series, so it was a nice surprise to find his name all over the place whenever I was searching for rigid body Dynamics.

Physac a 2D physics header-only library in C by Victor Fisac

Whenever I'm doing something new, I always feel more confident when I have the code of someone else I can read along with and digest. While doing anything in a new language and one that was so different from what I was used to, Physac helped me to double-check everything I was doing and make sure it made sense.

Everything by Erin Catto

I don't really know where to start when linking things from Erin Catto. If you have worked for a while in games, probably you already know about Box2D and even maybe heard about Erin. What you may not know is that Box 2D is not at all as complicated as I thought it was! By the end of my work with the physics engine, my main source of reference was the Box2D and the Box2D Lite source code. I didn't even realize that the intention behind **Box2D Lite** was to teach people on how to do physics simulations. Anyway, all the GDC presentations and all the code that Erin has written is excellent, and you should check it out.

Solving Rigid Body Contacts by Richard Tonge

At some point I got stuck with a weird behaviour we were having with multiple collisions in a single frame, I may talk a little more about it in a later post, but this presentation saved my ass. And now I'm forever a fan of Antonio Signorini